

T5Gemma 2 vs. Qwen 2.5 on TReB: Interim Report

What we set out to measure, what we actually found

akumch @ DiffusionTableQA

2026-04-20

TL;DR

We set out to compare an encoder-decoder architecture (T5Gemma 2 4B-4B, SFT'd on 100K FLAN) against a decoder-only at matched parameter count (Qwen 2.5 7B Instruct) on the [TReB](#) table-reasoning benchmark. The architecture comparison is **not delivered in this report**. Three reasons:

1. A transformers-v5 implementation bug in T5Gemma2 caps practical inference at **4094 input tokens** at `batch_size=1` (much less than the model's advertised 128K context). Filed as [hugging-face/transformers#45521](#); also filed a feature request for FA2 support as [#45522](#).
2. Our FLAN-SFT'd adapter **catastrophically mode-collapsed**: on every probed input it emits a single scalar JSON (`{"answer": 10}`, `{"answer": 100}`, etc.) followed by a repetition loop to `max_new_tokens`. Not a prompt-template issue — happens on plain free-form "Q: 2+2? A:" too.
3. Base T5Gemma 2 (no SFT) produces **empty outputs** on most tasks — expected, since Google only released the pretrained checkpoint.

Against that, **Qwen 2.5 7B Instruct ran the full 7,789-sample English split cleanly via vLLM** in ~7 minutes; local metrics landed (EM 0.159, ROUGE-L 0.309, numeric-match 0.434). The infrastructure works; the T5Gemma side of the comparison is the bottleneck.

1 Setup

Dataset. JT-LM/JIUTIAN-TReB (7,789 English samples across 26 tasks). TCoT (textual chain-of-thought) mode only; PoT and ICoT deferred.

Variants. `t5gemma_base` (pretrained), `t5gemma_sft` (our LoRA adapter at Hub commit `abae90a8`), `qwen_7b_instruct` (at native 32K context); a matched-4K Qwen run is planned.

Infrastructure. Custom eval harness (`eval.py`) with backend dispatch (vLLM for Qwen, HF + optional PEFT merge for T5Gemma). DeepSeek V3 via OpenRouter for LLM-as-Judge (12-class failure-mode taxonomy). Pods on RunPod H100 NVL; auto-termination on completion via per-variant `.done` sentinels + a background monitor (`monitor_pod.sh`).

Pipeline hardening (shipped during this session).

- **Incremental writes:** `predictions.flush()+fsync()`'d to `predictions.partial.jsonl` every 5 batches. Saved 95/100 once after a crash; previously lost all in-flight work.
- **Token-budgeted prompt assembly:** `build_prompt_tcot` encodes head (instruction/title) and tail (question + format instruction) separately, budgets table with remainder. Previously, right-truncation at a 2K cap silently dropped the question on 12% of samples — 1-in-8 T5Gemma predictions were literally not answering the question.

- **Adapter-merge safety:** `merge_adapter.py` now defaults `-base` from `adapter_config.json:base_model_name_or_path` instead of a hand-typed model id. `PeftModel.from_pretrained` doesn't shape-validate against the original base — naming mismatches silently produce junk weights.

2 Finding 1: transformers-v5 T5Gemma2 is capped at 4K, not 128K

Bisected with a targeted 9-sample diagnostic spanning 2.5K–25K tokens at `batch_size=1`:

input tokens	result	tensor b dim
2,497	OK	—
3,525	OK	—
5,015	FAIL	5,018
6,499	FAIL	6,502
7,493	FAIL	7,496
9,997	FAIL	10,000
14,967	FAIL	14,970
19,808	FAIL	19,811
25,135	FAIL	25,138

Every failure raises `RuntimeError: size of tensor a (4097) must match tensor b (N) at non-singleton dimension 3`, where `tensor a (4097)` is constant across every failure, and `tensor b = input_length + 3`. $4097 = 4 \times \text{sliding_window}(1024) + 1$ — the signature of a pre-allocated 4-window mask buffer that is not being resized for longer inputs. Affects both `eager` and `sdpa` attention paths; `flash_attention_2` is explicitly unsupported for `T5Gemma2ForConditionalGeneration`. The effective context ceiling is 4,094 tokens — vs. the 128K advertised on the [model card](#). Filed upstream.

3 Finding 2: our FLAN-SFT adapter mode-collapsed

Five probes on the merged SFT adapter (`abae90a8`), via an interactive REPL (`play.py`):

Prompt style	Task	Gold	Model output
TCoT	Code_Generation	Python function	<code>{"answer": 1}{ "answer": 2}...{"answer": 125}</code>
TCoT	Robustness_Evaluation	neutral	<code>{"answer": 10}</code> → loop reproducing the prompt
TCoT	Mathematical_Reasoning	\$101	<code>{"answer": 100}</code> → loop reproducing the prompt
Plain	“Q: What is 2+2? A:”	4	22 + 22 = 44 (repeated ~100×)
Plain	word problem, no TCoT	\$101	\$1.15 and \$1.10 (repeated ~100×)

Training loss (from the run-1 `trainer_state.json` on Hub) went $3.22 \rightarrow 1.47$ over 782 steps (batch 128, 100K FLAN, 1 epoch). Not random — real learning happened — but plateaued higher than our 50-step smoke overfit on 200 samples (loss 0.32). Combined with the inference behavior, the adapter has a specific failure pattern:

- Learned the *shape* of FLAN’s dominant target (short scalar JSON answer).
- Did not learn content: no multi-step reasoning, even on in-distribution arithmetic.
- Did not learn to emit EOS: every output runs to `max_new_tokens`, looping on an output it has already produced. Suggests FLAN training targets were ingested without an EOS-termination signal.

The *numbers* the model picks carry weak task-linkage (100 for a \$101 problem; 10 for a 3-way classification is random), so the forward pass isn’t completely decoupled from input — it’s the decoder loop that’s collapsed.

4 Finding 3: the pipeline works for Qwen — and has upstream knobs to trust

Qwen 2.5 7B Instruct, full 7,789 samples via vLLM at native 32K context, `batch_size=8` internal, `max_new_tokens=2048`. Wall-clock: ~ 7 min on $1 \times \text{H100 NVL}$; cost: $\sim \$1$.

Metric	Overall	Best task	Worst
Exact Match	0.159	Table_Fact_Checking 0.914	Instruction_Following (ref-free)
ROUGE-L	0.309	Table_Fact_Checking 0.914	Table_Column_Naming 0.028
Numeric Match	0.434	—	—

Raw local metrics undersell Qwen. Per spot-checks, Qwen emits multi-paragraph step-by-step reasoning but doesn't wrap the final answer in the requested `{"answer": ...}` envelope — tasks with EM=0 and ROUGE=0.3-0.5 (Understanding, Title_Naming, Summary) are format-not-capability failures. The *judge's* structured failure-mode classification (FORMAT, PARTIAL, CORRECT) will disentangle this. Judge run deferred until SFT is replaced.

5 Where this goes next

T5Gemma side. Our adapter needs to be redone. Likely improvements, in priority order:

1. Audit FLAN tokenization: ensure training targets are wrapped with a proper EOS token. Suspect current training let the model lose its ability to stop.
2. Use a dataset with longer, multi-step targets (e.g. FLAN-CoT, StrategyQA, GSM8K reasoning traces) so the model is tuned to *reason* rather than *emit JSON*.
3. More epochs — 1 epoch at this scale likely under-trained; eval loss 1.38 is higher than expected.
4. Longer input context in training (currently capped at 1024); won't be unlocked for inference until #45521 is fixed upstream, but doesn't hurt to train for.

Comparison that IS deliverable today. Even without a usable SFT, we can run `t5gemma_base` vs. `qwen@4K` (matched context) on the 100 stratified samples already run. It won't answer "does SFT close the gap?", but it will answer "does the architecture matter before either is instruction-tuned?" — a weaker but honest result.

Upstream dependency. Both transformers issues (#45521, #45522) need to land before T5Gemma 2's advertised capabilities are reachable via HF. Gemma family precedent (#37219, #35290, #31931, #41875, vllm#14881) has been resolved on 1–6 week timelines historically.

Appendix: key commits & pointers

Project repo (public)	https://github.com/junos-ai-org/jiutian-treb/tree/experiment-setup/experiments/t5gemma
Upstream bug	https://github.com/huggingface/transformers/issues/45521
Upstream FA2 request	https://github.com/huggingface/transformers/issues/45522
Minimal bug repro	<code>experiments/t5gemma_vs_qwen_treb/insights/transformers_bug_repro.py</code>
Eval driver	<code>experiments/t5gemma_vs_qwen_treb/eval.py</code>
Interactive REPL	<code>experiments/t5gemma_vs_qwen_treb/play.py</code>
Pod monitor	<code>experiments/t5gemma_vs_qwen_treb/monitor_pod.sh</code>
Qwen results (7,789)	<code>results/qwen_7b_instruct/predictions.jsonl</code> (local, gitignored)
T5Gemma base (100)	<code>results/t5gemma_base/predictions.jsonl</code>
Training log	<code>DiffusionTableQA/t5gemma-2-4b-flan-sft-ddp</code> Hub repo, commit <code>abae90a8</code>